

✓ Homework 4 Part 1

This is an individual assignment.

Write your answers using markdown cells or embed handwritten answers with

`IPython.display.Image`.

✓ Exercise 1 (8 points)

Consider the following application scenarios. Between the Linear Support Vector Machines (SVM) and Logistic Regression (LR), which one would you select for each scenario? Justify your answers.

1. (2 points) **Detecting fraudulent transactions in a credit card dataset, where the fraudulent transactions might be a minority class and are likely to be distinct from normal transactions.**

Answer:

I would select **Linear SVM** for this scenario.

Justification:

- Linear SVM is particularly effective when classes are well-separated and distinct, which is likely the case for fraudulent vs normal transactions
- SVM maximizes the margin between classes, making it robust to outliers and better at finding a clear decision boundary
- SVM's focus on support vectors (boundary cases) makes it effective even with imbalanced datasets where fraudulent transactions are minority
- While the class imbalance is a consideration, Linear SVM with appropriate class weights can handle this effectively
- SVM is less affected by the imbalanced nature since it focuses on the decision boundary rather than probability estimation

2. (2 points) **Text classification tasks, where each word or n-gram can be considered as a feature, leading to high-dimensional feature spaces.**

Answer:

I would select **Linear SVM** for this scenario.

Justification:

- Text classification with word/n-gram features creates very high-dimensional feature spaces (thousands to millions of features)
- Linear SVM is particularly well-suited for high-dimensional data and often achieves excellent performance in text classification tasks
- In high-dimensional spaces, data tends to be linearly separable, making linear SVM's margin maximization approach very effective
- SVM is computationally efficient in high dimensions compared to other methods
- Linear SVM is less prone to overfitting in high-dimensional spaces due to its margin-based formulation and structural risk minimization principle
- Text data is typically sparse, and SVM handles sparse high-dimensional data very efficiently

3. (2 points) **Medical diagnosis, where the occurrence of a rare disease is much lower than that of the healthy population.**

Answer:

I would select **Logistic Regression** for this scenario.

Justification:

- Medical diagnosis requires probability estimates for informed decision-making, not just binary classifications
- Logistic Regression naturally provides calibrated probability outputs (probability of disease), which is crucial for medical applications where doctors need confidence levels
- These probabilities help establish diagnostic thresholds and allow for risk assessment
- Medical professionals can use probability outputs to make more nuanced decisions (e.g., "85% probability of disease" vs "15% probability")
- Logistic Regression is more interpretable, allowing doctors to understand which features contribute to the diagnosis
- While class imbalance exists, Logistic Regression can handle this with appropriate techniques (class weights, threshold adjustment)

- The probabilistic output is essential for cost-sensitive decisions in healthcare (balancing false positives vs false negatives)
4. (2 points) **Large-scale text categorization, where the focus is on assigning documents to predefined categories, rather than estimating the probability of belonging to a specific class.**

Answer:

I would select **Linear SVM** for this scenario.

Justification:

- The problem explicitly states that the focus is on category assignment rather than probability estimation, which is exactly what SVM is designed for
 - Linear SVM directly optimizes for the decision boundary without computing probabilities, making it more efficient for large-scale problems
 - SVM's margin maximization leads to better generalization on unseen documents
 - For large-scale text categorization, computational efficiency is critical - Linear SVM is faster than Logistic Regression since it doesn't need to compute and normalize probabilities
 - Text categorization typically involves high-dimensional sparse features (similar to question 2), where Linear SVM excels
 - SVM focuses computational resources on support vectors (boundary cases), making it scalable for large datasets
 - Since probability outputs are not needed, the computational overhead of Logistic Regression's probability calibration is unnecessary
-

✓ Exercise 2 (10 points)

Consider a feed-forward multilayer perceptron with a D - M - D architecture ($M \leq D$) and linear activation functions. Show that such a neural network, when trained to predict the input at the output layer, performs principal component analysis by considering the minimization it solves. Indicate if the network will require bias terms.

Answer:

Network Architecture:

- Input layer: D neurons (input $\mathbf{x} \in \mathbb{R}^D$)

- Hidden layer: M neurons (where $M \leq D$), with hidden representation $\mathbf{h} \in \mathbb{R}^M$
- Output layer: D neurons (output $\hat{\mathbf{x}} \in \mathbb{R}^D$)
- All activation functions are linear

Forward Pass:

Let $\mathbf{W}_1 \in \mathbb{R}^{M \times D}$ be the input-to-hidden weights and $\mathbf{W}_2 \in \mathbb{R}^{D \times M}$ be the hidden-to-output weights.

Without bias terms:

$$\begin{aligned}\mathbf{h} &= \mathbf{W}_1 \mathbf{x} \\ \hat{\mathbf{x}} &= \mathbf{W}_2 \mathbf{h} = \mathbf{W}_2 \mathbf{W}_1 \mathbf{x}\end{aligned}$$

Objective Function:

The network is trained to reconstruct the input, minimizing:

$$E = \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - \hat{\mathbf{x}}_n\|^2 = \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - \mathbf{W}_2 \mathbf{W}_1 \mathbf{x}_n\|^2$$

Connection to PCA:

PCA finds an M -dimensional subspace that minimizes reconstruction error. The principal components are the eigenvectors corresponding to the M largest eigenvalues of the data covariance matrix.

For the linear autoencoder:

- The hidden layer $\mathbf{h} = \mathbf{W}_1 \mathbf{x}$ projects the input into an M -dimensional space
- The output layer $\hat{\mathbf{x}} = \mathbf{W}_2 \mathbf{h}$ reconstructs the input from this lower-dimensional representation
- The combined transformation $\mathbf{W}_2 \mathbf{W}_1$ represents a projection matrix onto an M -dimensional subspace

At the optimal solution, the columns of $\mathbf{W}_2^T$ span the same subspace as the first M principal components. The network learns to:

1. Encode the input into the M -dimensional subspace that captures maximum variance
2. Decode from this subspace to reconstruct the original input

This is exactly what PCA does: find the M -dimensional linear subspace that minimizes reconstruction error, which is equivalent to maximizing retained variance.

Bias Terms:

The network does **NOT require bias terms**. PCA operates on mean-centered data and finds directions of maximum variance without needing an offset. If the data is pre-centered (zero mean), bias terms would be unnecessary and would remain zero

during training. The linear projection and reconstruction can be achieved purely through the weight matrices $\mathbf{W}_1$ and $\mathbf{W}_2$.

✓ Exercise 3 (6 points)

Compare and contrast the objective functions of PCA and Fisher Linear Discriminant Analysis (FLDA) for dimensionality reduction. Specifically, explain how FLDA incorporates class information that PCA ignores, defining all terms such as within-class scatter and between-class scatter matrices. And, discuss the advantages and disadvantages of each method, particularly in the context of supervised vs unsupervised learning.

Answer:

✓ Objective Functions:

PCA (Unsupervised):

- **Goal:** Maximize variance in the projected data
- **Objective:** Find directions that maximize $\mathbf{w}^T \mathbf{S} \mathbf{w}$, where $\mathbf{S}$ is the total data covariance matrix

$$\mathbf{S} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T$$

- PCA finds eigenvectors of $\mathbf{S}$ corresponding to largest eigenvalues
- **Does NOT use class labels** - purely based on data variance

FLDA (Supervised):

- **Goal:** Maximize between-class separation while minimizing within-class scatter
- **Objective:** Maximize the Fisher criterion (ratio):

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

where:

Within-class scatter matrix $\mathbf{S}_W$: Measures scatter of samples around their respective class means

$$\mathbf{S}_W = \sum_{c=1}^C \sum_{\mathbf{x}_n \in \mathcal{C}_c} (\mathbf{x}_n - \boldsymbol{\mu}_c)(\mathbf{x}_n - \boldsymbol{\mu}_c)^T$$

where $\boldsymbol{\mu}_c$ is the mean of class c and $\mathcal{C}_c$ is the set of samples in class c

Between-class scatter matrix $\mathbf{S}_B$: Measures separation between class means

$$\mathbf{S}_B = \sum_{c=1}^C N_c (\boldsymbol{\mu}_c - \bar{\mathbf{x}})(\boldsymbol{\mu}_c - \bar{\mathbf{x}})^T$$

where N_c is the number of samples in class c , and $\bar{\mathbf{x}}$ is the overall mean

- FLDA finds directions by solving the generalized eigenvalue problem:

$$\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w}$$

Key Difference - Use of Class Information:

FLDA incorporates class labels by:

1. Computing separate statistics for each class (class means $\boldsymbol{\mu}_c$)
2. Maximizing the separation between these class means (via $\mathbf{S}_B$)
3. Minimizing the spread within each class (via $\mathbf{S}_W$)
4. Finding projections that make classes as distinguishable as possible

PCA ignores class labels by:

1. Treating all data points equally regardless of class
2. Only considering overall data variance
3. May project data onto directions with high variance but poor class separation

Advantages and Disadvantages:

PCA Advantages:

- **Unsupervised:** Works without labeled data
- **General purpose:** Good for visualization, noise reduction, and general dimensionality reduction
- **Captures overall data structure:** Preserves maximum variance
- **No class limitation:** Works with any number of classes or no classes

PCA Disadvantages:

- **Ignores class information:** High-variance directions may not separate classes well
- **Not optimized for classification:** May discard discriminative information in low-variance directions
- **Can be misleading for supervised tasks:** Important class boundaries might be in low-variance directions

FLDA Advantages:

- **Optimized for classification:** Directly maximizes class separability
- **Supervised learning:** Uses label information effectively
- **Better for discrimination:** Often achieves better classification performance with fewer dimensions

- **Focuses on discriminative features:** Finds features that best separate classes

FLDA Disadvantages:

- **Requires labeled data:** Cannot be used in unsupervised settings
 - **Limited dimensions:** Can extract at most $C - 1$ features (where C is number of classes)
 - **Assumes class distributions:** Performance depends on within-class and between-class structure
 - **Can fail with small samples:** $\mathbf{S}_W$ must be invertible; problematic when $N < D$ or classes overlap significantly
-

✓ Exercise 4 (10 points)

Design the simplest feed-forward MLP, by specifying numerical values for its weights and biases, using the threshold activation function,

$$\phi(x) = \begin{cases} 1 & x > 0 \\ -1 & x \leq 0 \end{cases}, \text{ to solve the following scenarios:}$$

1. (2 points) **The NAND logic gate, $\overline{x_1 \cap x_2} = \overline{x_1} \cup \overline{x_2}$, where $x_i \in \{0, 1\}, i = 1, 2$.**

Answer:

The NAND gate outputs 0 only when both inputs are 1, otherwise outputs 1.

x_1	x_2	NAND output
0	0	1
0	1	1
1	0	1
1	1	0

MLP Architecture:

- Input layer: 2 neurons (x_1, x_2)
- Hidden layer: None needed (single layer perceptron)
- Output layer: 1 neuron with threshold activation

Solution:

Using threshold activation $\phi(x) = \begin{cases} 1 & x > 0 \\ -1 & x \leq 0 \end{cases}$

We need to map $\{0,1\}$ outputs to $\{1,-1\}$ format. Let's design the network:

Weights and Bias:

- $w_1 = -1$ (weight for x_1)
- $w_2 = -1$ (weight for x_2)
- $b = 1$ (bias)

Network equation:

$$y = \phi(w_1x_1 + w_2x_2 + b) = \phi(-x_1 - x_2 + 1)$$

Verification:

- $(x_1, x_2) = (0, 0)$: $\phi(-0 - 0 + 1) = \phi(1) = 1 \checkmark$ (NAND = 1)
- $(x_1, x_2) = (0, 1)$: $\phi(-0 - 1 + 1) = \phi(0) = -1 \rightarrow$ Need adjustment

Let me reconsider with proper threshold adjustment:

Corrected Solution:

- $w_1 = -2$
- $w_2 = -2$
- $b = 3$

$$y = \phi(-2x_1 - 2x_2 + 3)$$

Verification:

- $(0, 0)$: $\phi(3) = 1 \checkmark$
- $(0, 1)$: $\phi(1) = 1 \checkmark$
- $(1, 0)$: $\phi(1) = 1 \checkmark$
- $(1, 1)$: $\phi(-1) = -1 \rightarrow$ maps to 0 $\checkmark$

2. (4 points) **The following two-dimensional dataset,**

Answer:

Note: The two-dimensional dataset image is not visible in the text. I'll provide a general approach for designing an MLP for a typical 2D classification problem.

For a 2D dataset with non-linearly separable classes, the approach would be:

MLP Architecture:

- Input layer: 2 neurons (x_1, x_2)
- Hidden layer: 2-4 neurons (depending on complexity)
- Output layer: 1 neuron

General Approach:

If the problem requires XOR-like separation:

- Hidden layer: 2 neurons
- Example weights for XOR pattern:

Hidden Layer:

- Neuron 1: $h_1 = \phi(w_{11}x_1 + w_{12}x_2 + b_1)$ with $w_{11} = 1, w_{12} = 1, b_1 = -0.5$
- Neuron 2: $h_2 = \phi(w_{21}x_1 + w_{22}x_2 + b_2)$ with $w_{21} = 1, w_{22} = 1, b_2 = -1.5$

Output Layer:

- $y = \phi(v_1 h_1 + v_2 h_2 + b_o)$ with $v_1 = 1, v_2 = -1, b_o = -0.5$

If you can provide the specific dataset image or description, I can give exact weights and biases for that particular problem.

3. (4 points) The given ground truth table,

x1	x2	x3	t
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Answer:

Analyzing the truth table:

x_1	x_2	x_3	t
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Pattern Recognition: Looking at when $t = 1$: This is a 3-input parity function (odd parity) - output is 1 when odd number of inputs are 1.

- Row 2: one 1 $\rightarrow t = 1$ ✓
- Row 3: one 1 $\rightarrow t = 1$ ✓
- Row 5: one 1 $\rightarrow t = 1$ ✓
- Row 8: three 1s $\rightarrow t = 1$ ✓

This is $t = x_1 \oplus x_2 \oplus x_3$ (3-bit XOR)

MLP Architecture:

- Input layer: 3 neurons
- Hidden layer: 4 neurons (needed for 3-input XOR)
- Output layer: 1 neuron

Design Strategy:

The 3-input XOR can be decomposed as: $(x_1 \oplus x_2) \oplus x_3$

Hidden Layer (4 neurons):

Neuron h_1 : Detects x_1 AND x_2

- Weights: $w_{11} = 2, w_{12} = 2, w_{13} = 0$, bias $b_1 = -3$
- $h_1 = \phi(2x_1 + 2x_2 - 3) \rightarrow$ outputs 1 only when $x_1 = x_2 = 1$

Neuron h_2 : Detects x_1 OR x_2

- Weights: $w_{21} = 2, w_{22} = 2, w_{23} = 0$, bias $b_2 = -1$
- $h_2 = \phi(2x_1 + 2x_2 - 1) \rightarrow$ outputs 1 when at least one of x_1, x_2 is 1

Neuron h_3 : Detects $(x_1 \oplus x_2)$ AND x_3

- Weights: $w_{31} = 2, w_{32} = 2, w_{33} = 4$, bias $b_3 = -3$
- This is complex, let me use a cleaner approach:

Simplified Architecture:

Hidden Layer:

- $h_1 = \phi(2x_1 + 2x_2 - 1)$ with weights $(2, 2, 0, -1) \rightarrow x_1$ OR x_2
- $h_2 = \phi(2x_1 + 2x_2 - 3)$ with weights $(2, 2, 0, -3) \rightarrow x_1$ AND x_2
- $h_3 = \phi(2x_3 - 1)$ with weights $(0, 0, 2, -1) \rightarrow x_3$
- $h_4 = \phi(2h_1 - 4h_2 + 2x_3 - 1) \rightarrow$ XOR of first two and x_3

Output Layer:

- $y = \phi(2h_1 - 4h_2 + 2h_3 - 1)$

Complete weight specification:

Input to Hidden (3×4 weight matrix + 4 biases):

```
h1: w=[2, 2, 0], b=-1
h2: w=[2, 2, 0], b=-3
h3: w=[0, 0, 2], b=-1
h4: w=[2, -4, 2], b=-1 (takes x1 OR x2, x1 AND x2, x3)
```

Hidden to Output:

```
y: v=[2, -4, 2, 0], b=-1
```

This implements the 3-bit XOR function using the threshold activation function.

✓ Exercise 5 (4 points)

Explain why the input-to-hidden weights must be different from each other (e.g., random) or else learning cannot proceed well. Specifically, what happens if the weights are initialized so as to have identical values?

Answer:

Input-to-hidden weights must be different (typically randomized) because **identical weight initialization creates a symmetry problem** that prevents effective learning.

✓ What Happens with Identical Weights:

If all input-to-hidden weights are initialized identically (e.g., all weights $w_{ij}^{(1)} = c$ for some constant c):

1. All hidden neurons compute the same function:

- Each hidden neuron receives: $z_j = \sum_i w_{ij}x_i + b_j$
- If all w_{ij} are identical and biases are identical, then $z_1 = z_2 = \dots = z_M$
- All hidden neurons produce the same activation: $h_1 = h_2 = \dots = h_M$

2. Identical gradients during backpropagation:

- During backpropagation, all hidden neurons receive the same error signal
- Weight updates are computed as: $\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$
- Since all neurons have identical inputs and outputs, all gradients are identical
- Therefore, all weights receive identical updates: $\Delta w_{1j} = \Delta w_{2j} = \dots$

3. Symmetry is never broken:

- After each update: $w_{ij}^{new} = w_{ij}^{old} - \eta \Delta w_{ij}$
- Since all weights start identical and receive identical updates, they remain identical forever
- The hidden neurons never specialize or learn different features

4. Network becomes equivalent to a single hidden neuron:

- With M identical hidden neurons, the network effectively has only **one unique hidden unit repeated M times**
- This drastically reduces the network's representational capacity

- The network cannot learn complex non-linear functions

Why Random Initialization Works:

- **Breaks symmetry:** Different initial weights → different neuron activations → different gradients
- **Allows specialization:** Each hidden neuron can learn to detect different features
- **Full representational capacity:** All M hidden neurons contribute independently
- **Enables diverse feature learning:** Different neurons capture different aspects of the input patterns

Conclusion:

Random weight initialization is essential for **symmetry breaking** in neural networks. Without it, hidden neurons would be redundant copies, defeating the purpose of having multiple hidden units and severely limiting the network's learning capability.

✓ Exercise 5 (7 points)

Answer the following questions, and provide appropriate justifications, about Convolutional Neural Networks (CNNs):

1. (2 points) **Would you prefer to add more filters in the first convolutional layer or the second convolutional layer?**

Answer:

I would prefer to **add more filters in the second convolutional layer** rather than the first.

Justification:

1. Feature Hierarchy:

- The first convolutional layer detects simple, low-level features (edges, corners, basic textures)
- The second convolutional layer combines these low-level features to detect more complex, higher-level patterns

- More complex patterns require more diverse representations, hence more filters

2. Computational Efficiency:

- The first layer operates on the full input resolution (larger spatial dimensions)
- Adding filters to the first layer increases computation significantly since each filter processes every pixel
- The second layer typically operates on reduced spatial dimensions (due to pooling), making additional filters computationally cheaper

3. Information Compression:

- Early layers should focus on capturing essential low-level features efficiently
- Later layers need more capacity to represent the combinatorial explosion of higher-level feature combinations
- A smaller number of low-level features can be combined in many ways to create numerous high-level features

4. Common CNN Architecture Practice:

- Standard CNN architectures (VGG, ResNet, etc.) typically increase the number of filters as depth increases
- Pattern: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ filters is more common than the reverse
- This follows the principle of building increasingly abstract representations

5. Representational Power:

- The second layer has more combinations to represent since it builds on features from the first layer
- More filters in deeper layers provide greater capacity for learning complex, task-specific features

2. (2 points) Why would you want to add a max pooling layer rather than a convolutional layer with the same stride?

Answer:

I would prefer **max pooling** over a convolutional layer with the same stride for several important reasons:

Advantages of Max Pooling:

1. No Parameters to Learn:

- Max pooling has **zero learnable parameters**

- A convolutional layer with stride has weights and biases that need to be learned
- Fewer parameters → less risk of overfitting, faster training, smaller model size

2. Invariance to Small Translations:

- Max pooling provides **local translation invariance**
- It selects the maximum activation within a region, making the representation robust to small shifts
- A conv layer with stride might miss features that are slightly offset

3. Computational Efficiency:

- Max pooling is a simple selection operation (just finding maximum)
- No multiply-accumulate operations needed
- Convolutional layers require expensive convolution operations even with stride

4. Feature Enhancement:

- Max pooling amplifies the strongest activations
- Helps the network focus on the most prominent features
- Acts as a form of **feature selection** by keeping only the most relevant information

5. Reduces Overfitting:

- By providing a form of regularization through aggressive downsampling
- Forces the network to learn more robust features
- The discretization effect of max pooling adds noise resistance

6. Separates Feature Extraction from Downsampling:

- Clear architectural separation: convolution learns features, pooling reduces dimensions
- This modularity makes the network easier to interpret and design
- Strided convolutions conflate these two operations

When Strided Convolution Might Be Better:

- When you need learnable downsampling (e.g., learning which information to preserve)
- In very deep networks where every operation should be learnable
- In architectures like fully convolutional networks where differentiability throughout is important

Conclusion: For most traditional CNN architectures, max pooling is preferred because it provides spatial downsampling without adding parameters, improves

translation invariance, and acts as a feature selector—all while being computationally efficient.

3. (3 points) **Consider a CNN composed of three convolutional layers, each with 3×3 kernels, a stride of 2, and "same" padding. The lowest layer outputs 100 feature maps, the middle one outputs 200, and the top one outputs 400. The input images are RGB images of 200×300 pixels. What is the total number of parameters in the CNN?**

Answer:

Given:

- Input: RGB images of 200×300 pixels (3 channels)
- Layer 1: 3×3 kernels, stride 2, "same" padding, 100 output feature maps
- Layer 2: 3×3 kernels, stride 2, "same" padding, 200 output feature maps
- Layer 3: 3×3 kernels, stride 2, "same" padding, 400 output feature maps

Calculating Parameters for Each Layer:

Layer 1:

- Input channels: 3 (RGB)
- Output feature maps: 100
- Kernel size: 3×3
- Parameters per filter: $3 \times 3 \times 3 = 27$ weights
- Bias per filter: 1
- Total parameters: $100 \times (27 + 1) = 100 \times 28 = 2,800$

Layer 2:

- Input channels: 100 (from Layer 1)
- Output feature maps: 200
- Kernel size: 3×3
- Parameters per filter: $3 \times 3 \times 100 = 900$ weights
- Bias per filter: 1
- Total parameters: $200 \times (900 + 1) = 200 \times 901 = 180,200$

Layer 3:

- Input channels: 200 (from Layer 2)
- Output feature maps: 400
- Kernel size: 3×3
- Parameters per filter: $3 \times 3 \times 200 = 1,800$ weights
- Bias per filter: 1
- Total parameters: $400 \times (1,800 + 1) = 400 \times 1,801 = 720,400$

Total Number of Parameters:

$$\text{Total} = 2,800 + 180,200 + 720,400 = \boxed{903,400 \text{ parameters}}$$

Formula for Convolutional Layer Parameters:

For a convolutional layer with:

- C_{in} input channels
- C_{out} output filters
- Kernel size $K \times K$

$$\text{Parameters} = C_{out} \times (K \times K \times C_{in} + 1)$$

where the "+1" accounts for the bias term per filter.

✓ On-Time (4 points) + Notebook PDF (1 point)

Submit your Notebook PDF before the deadline.

Submit Your Solution

Confirm that you've successfully completed the assignment.

Along with the Notebook, include a PDF of the notebook with your solutions.

`add` and `commit` the final version of your work, and `push` your code to your GitHub repository.

Submit the URL of your GitHub Repository as your assignment submission on Canvas.
